

Object Oriented Programming (OOP)

Semester: 2nd

Credit Hour: (01)

Department of Computer Science

Course Instructor: Engr. Zakria Bacha

Lab Task No. 13

OOP in C++

Previous Lab:

In previous lab, we have covered below three topics:

1. Pure Virtual Function
2. Abstract Class

Current Topics for Lab:

1. Association
2. Composition Vs Aggregation

Association:

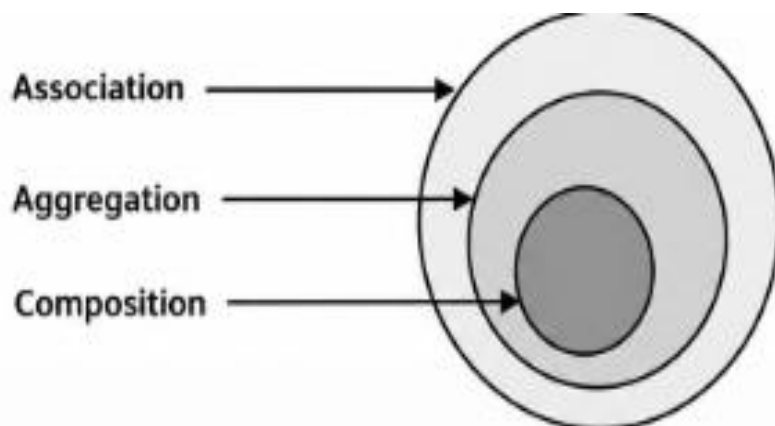
Association refers to a relationship or connection between two or more things. The term is used in several different contexts:

Explanation:

These real-world entities are not isolated because they talk to each other, use each other, and relate to each other. That is exactly what association captures in code.

Real World	Programming
A school system	Classes: Teacher, Student, Course
A hospital system	Classes: Doctor, Patient, Appointment
A car rental app	Classes: Driver, Car, Booking

Below Figure represent each subject.



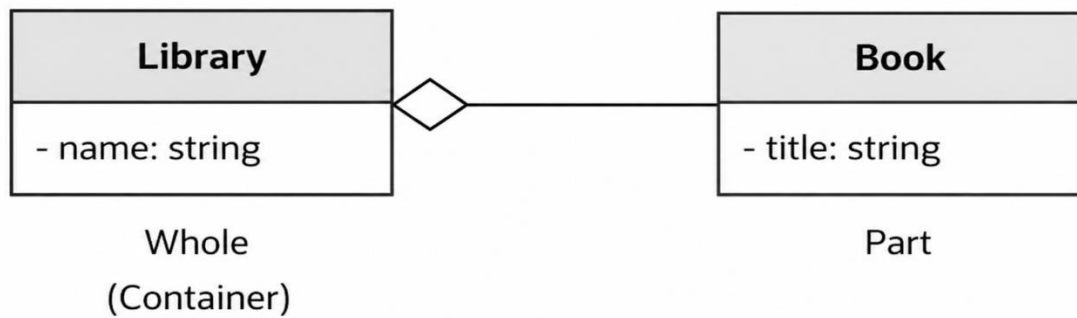
Example via Code:

```
6 class Student {
7 public:
8     string name;
9     Student(string n)
10    {
11        name = n;
12    }
13 };
14
15 class Teacher {
16 public:
17     string name;
18     Teacher(string n)
19    {
20        name = n;
21    }
22
23     void teach(Student s)
24    {
25        cout<<name<<" is teaching "<<s.name<<endl;
26    }
27 };
28
29 int main() {
30     Student s("sameen");
31     Teacher t("Sir Ahmad");
32     ✓ t.teach(s); // Teacher uses Student
33     return 0;
34 }
```

Aggregation:

A “whole part” relationship where the part can exist independently of the whole. It signifies a weaker relationship compared to composition.

National University of Computer and Emerging Sciences
Peshawar Campus



Composition:

A strong “whole part” relationship where the part cannot exist independently of the whole.

The lifecycle of the part is tied to the life cycle of the whole.

Code for Aggregation: To create an aggregation relationship between department and teacher, follow the below code as shown in Figure 1.

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Teacher
5  {
6  private:
7      int id;
8      string name;
9      string subject;
10
11 public:
12     Teacher(int i, string n, string s)
13     {
14         id = i;
15         name = n;
16         subject = s;
17     }
18     void showTeacher()
19     {
20         cout<<"Teacher ID" <<id<<endl;
21         cout<<"Teacher Name"<<name<< endl;
22         cout<<"Subject"<<subject<<endl;
23     }
24 };
```

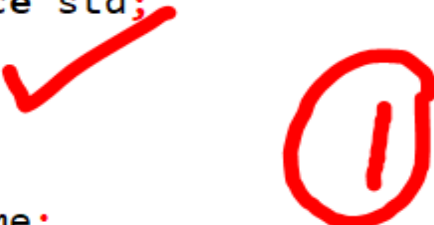


Figure 1

2

```
26 class Department
27 {
28     private:
29         string deptName;
30         Teacher* teacher; // Aggregation ✓
31
32     public:
33         Department(string d, Teacher* t)
34         {
35             deptName = d;
36             teacher = t; ✓
37         }
38
39         void showDepartment()
40         {
41             cout << "Department Name: " << deptName << endl;
42             teacher->showTeacher(); ✓
43         }
44     };
45
```

Code for Composition:

Let's take an example of Question and option scenario. The lifecycle of option depends on the Question. If question exist, then there will be also option. If no Question, so option will be not exist.

```
62 #include <iostream>
63 #include <string>
64 using namespace std;
65
66 class Option
67 {
68     private:
69         string optionText;
70
71     public:
72         Option(string text)
73         {
74             optionText = text;
75         }
76
77         void showOption()
78         {
79             cout << optionText << endl;
80         }
81     };
82
```



National University of Computer and Emerging Sciences
Peshawar Campus

```
83 class Question
84 {
85     private:
86         string statement;
87         Option op1;    // Composition
88         Option op2;    // Composition
89         Option op3;    // Composition
90         Option op4;    // Composition
91     public:
92         Question(string s, string a, string b, string c, string d)
93             : statement(s), op1(a), op2(b), op3(c), op4(d)
94         {
95         }
96
97         void display()
98         {
99             cout<<"Question: " << statement << endl;
100             cout<<"1. ";
101             op1.showOption();
102
103             cout<<"2. ";
104             op2.showOption();
105
106             cout<<"3. ";
107             op3.showOption();
108
109             cout<<"4. ";
110             op4.showOption();
111         }
112     };
113
114     int main()
115     {
116         Question q1(
117             "C++ is Programming Language",
118             "True",
119             "False",
120             "May True",
121             "May False"
122         );
123
124         q1.display();
125
126         return 0;
127     }
128 }
```


National University of Computer and Emerging Sciences

Peshawar Campus

Task 1:

A software company is developing a **Hospital Management System** where each patient admission must generate 1 medical record. Without admission, the medical record cannot exist. Implement this scenario using **Composition** in C++. Create a class MedicalRecord that stores disease name, room charges, and treatment cost, and calculates total charges. Create another class PatientAdmission that stores admission ID, patient name, and contains 1 object of MedicalRecord. Input data for 3 patients and display complete details with total bill.

Task 2:

A software company is developing a **University Management System** where 1 department can assign existing professors. A professor can exist independently and may teach in another department later. Implement this scenario using **Aggregation** in C++. Create a class Professor that stores professor ID, name, and specialization. Create another class Department that stores department name and uses a pointer/reference to a Professor object. Create data for 2 professors and assign 1 professor to a department, then display complete details.

=====

=====

National University of Computer and Emerging Sciences
Peshawar Campus